HARI 2

# Panduan lab hari 2

## *Building Modern Web Application*

Praktik Hari 2 melanjutkan proyek Express dan EJS dengan validasi formulir, basis data PostgreSQL di Supabase, Prisma ORM, penyimpanan data barang, dan komponen partial yang dipakai bersama.

### Urutan praktik


| Lab | Topik praktik | Halaman |
|---|---|---|
| 01 | Validasi formulir | 2 |
| 02 | Menyiapkan PostgreSQL di Supabase | 4 |
| 03 | Menghubungkan aplikasi dengan Prisma ORM | 5 |
| 04 | Operasi CRUD dasar | 6 |
| 05 | Komponen partial EJS | 9 |

LAB 01

# Validasi formulir

## Tujuan

Menerima data melalui metode POST dan memeriksa nama serta format email di sisi server menggunakan express-validator.

## Langkah praktik

1. Buka terminal pada proyek Hari 1, lalu instal pustaka validasi:

```
npm install express-validator
```

2. Di app.js, tambahkan impor berikut di dekat impor Express. Letakkan konfigurasi pembacaan formulir setelah const app = express(); dan sebelum semua rute:

```
const { body, validationResult } = require('express-validator');

app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));
```

Jika pengaturan view engine sudah ada dari Hari 1, gunakan pengaturan tersebut.

3. Buat views/form.ejs dan isi dengan kode berikut. Input email memakai tipe teks agar format yang salah tetap dapat dikirim untuk menguji validasi server:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Validasi formulir</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="container mt-5">
 <h2>Form tambah data</h2>
 <% if (typeof errors !== 'undefined' && errors.length) { %>
  <div class="alert alert-danger" role="alert">
   <ul class="mb-0">
    <% errors.forEach(err => { %>
     <li><%= err.msg %></li>
    <% }); %>
   </ul>
  </div>
 <% } %>
 <form action="/submit-form" method="POST">
  <div class="mb-3">
   <label for="nama" class="form-label">Nama</label>
   <input id="nama" type="text" name="nama" class="form-control">
  </div>
  <div class="mb-3">
   <label for="email" class="form-label">Email</label>
   <input id="email" type="text" name="email" class="form-control">
  </div>
  <button type="submit" class="btn btn-primary">Kirim</button>
 </form>
</body>
</html>
```

## Rute POST dan pengujian validasi

4. Tambahkan rute GET berikut di app.js untuk menampilkan formulir:

```
app.get('/form', (req, res) => {
 res.render('form');
});
```

5. Tambahkan rute POST beserta aturan validasinya setelah rute GET. trim() menghapus spasi di tepi nama, sehingga nama berisi spasi saja tetap ditolak:

```
app.post(
 "/submit-form",
 [
  body("nama").trim().notEmpty().withMessage("Nama tidak boleh kosong"),
  body("email").trim().isEmail().withMessage("Format email tidak valid"),
 ],
 (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
   return res.status(422).render("form", {
    errors: errors.array(),
   });
  }
  res.render("form-success", {
   nama: req.body.nama,
   email: req.body.email,
  });
 },
);
```

6. Buat halaman “form-success.ejs” pada views.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sukses</title>
  <link rel="stylesheet" href="/css/bootstrap.min.css">
</head>
<body>
  <h1>Formulir berhasil dikirim!</h1>
  <p>Nama: <%= nama %></p>
  <p>Email: <%= email %></p>
  <a href="/form" class="btn btn-primary">Kembali ke Form</a>
</body>
</html>
```

7. Simpan berkas, jalankan ulang server, lalu buka http://localhost:3000/form.

## Hasil pembelajaran

Rute GET menampilkan formulir. Rute POST membaca req.body, memvalidasi isinya, lalu menampilkan pesan kesalahan atau hasil yang berhasil diproses. Data formulir ini belum disimpan ke basis data.

LAB 02

# Menyiapkan PostgreSQL di Supabase

## Tujuan

Membuat basis data PostgreSQL di Supabase dan menyimpan alamat koneksinya sebagai variabel lingkungan pada proyek Node.js.

## Langkah praktik

1. Buka https://supabase.com/dashboard dan masuk ke akun Anda. Pilih organisasi, lalu New project.
2. Isi nama proyek, kata sandi database, dan region terdekat yang tersedia. Pilih paket Free bila tersedia, lalu tunggu proyek siap.

Periksa kuota dan kebijakan jeda pada dasbor. Untuk akses hanya melalui Prisma, nonaktifkan Data API pada pengaturan API proyek latihan.

3. Buka Connect, pilih Session pooler port 5432, lalu salin URI. Ganti kata sandi database; encode karakter khusus, misalnya @ menjadi %40. Tambahkan ?sslmode=require atau &sslmode=require jika URI sudah memiliki parameter. Project URL dan API key bukan URI database.

```
postgresql://postgres.iwkqbdmnrqhgtmiiwktp:Inipasword_123@aws-0-ap-southeast-
1.pooler.supabase.com:5432/postgres?sslmode=require
```

4. Kembali ke terminal proyek dan instal dotenv:

```
npm install dotenv
```

5. Buat .env sejajar dengan package.json. Isi PORT dan DATABASE_URL berikut, lalu ganti placeholder dengan URI Session pooler dari dasbor. Simpan URI sebagai satu baris; jangan menggandakan kunci:

```
PORT=3000
DATABASE_URL="GANTI_DENGAN_URI_SESSION_POOLER_SUPABASE"
```

6. Tambahkan baris berikut ke .gitignore agar konfigurasi rahasia dan dependensi tidak masuk ke Git:

```
.env
node_modules/
```

7. Ubah bagian awal app.js menjadi seperti berikut. Pertahankan konfigurasi EJS, middleware formulir, dan seluruh rute yang sudah dibuat:

```
require('dotenv').config();
const express = require('express');
const { body, validationResult } = require('express-validator');
const app = express();
const port = process.env.PORT || 3000;
```

Pemanggilan app.listen(port, ...) tetap berada di bagian akhir app.js. Jangan mencetak DATABASE_URL ke terminal atau menyertakannya dalam tangkapan layar.

## Uji hasil

Jalankan ulang node app.js, lalu buka /form untuk memastikan aplikasi tetap bekerja. Pemeriksaan koneksi basis data dilakukan melalui sinkronisasi skema pada Lab 3.

## Hasil pembelajaran

Proyek Supabase siap. PORT dan URI Session pooler tersimpan pada .env. dotenv dimuat sebelum konfigurasi aplikasi; koneksi database diuji pada Lab 3.

LAB 03

# Menghubungkan aplikasi dengan Prisma ORM

## Tujuan

Mendefinisikan model Barang dan menyinkronkannya dengan PostgreSQL Supabase menggunakan Prisma db push.

## Langkah praktik

1. Gunakan Prisma 6.19.0 agar konfigurasi dan impor CommonJS pada latihan konsisten. Instal CLI serta Client dengan versi yang sama, lalu inisialisasi Prisma:

```
npm install --save-dev --save-exact prisma@6.19.0
npm install --save-exact @prisma/client@6.19.0
npx prisma init --datasource-provider postgresql
```

2. Buka prisma/schema.prisma dan ganti isinya dengan skema lengkap berikut. Jangan menambahkan blok datasource atau generator kedua:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Barang {
  id       Int      @id @default(autoincrement())
  nama     String
  harga    Int
  stok     Int
  createdAt DateTime @default(now())
}
```

3. Jika inisialisasi membuat prisma.config.ts, ganti isinya dengan konfigurasi berikut. Jika belum ada, buat di folder utama proyek. Impor dotenv memastikan CLI membaca .env:

```
import 'dotenv/config';
import { defineConfig } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma'
});
```

4. Pastikan .env tetap berisi URI Session pooler Supabase port 5432 dengan sslmode=require. Jalankan validasi skema, sinkronisasi tabel, dan pembuatan Prisma Client:

```
npx prisma validate
npx prisma db push
```

## Uji hasil

Terminal menyatakan skema valid dan database sinkron. Jalankan npx prisma studio, lalu buka model Barang atau periksa melalui Table Editor Supabase. Tabel masih kosong. Akhiri Studio dengan Ctrl + C.

## Hasil pembelajaran

db push membentuk tabel latihan tanpa basis data shadow dan tanpa riwayat migrasi. Jika muncul peringatan kehilangan data, hentikan; jangan gunakan opsi pemaksaan. Proyek produksi memerlukan alur Prisma Migrate tersendiri.

Rujukan konfigurasi Prisma 6: https://www.prisma.io/docs/orm/v6/reference/prisma-config-reference

LAB 04

# Operasi CRUD dasar

## Tujuan

Menambahkan barang ke PostgreSQL dan menampilkan daftar barang melalui Prisma. Praktik ini mencakup Create dan Read sebagai dasar CRUD.

## Langkah praktik

1. Buat views/barang.ejs untuk menampilkan daftar barang. Isi dengan kode lengkap berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Manajemen barang</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="container mt-5">
 <h2>Manajemen barang</h2>
 <a href="/barang/tambah" class="btn btn-primary mb-3">Tambah barang</a>
 <div class="table-responsive">
  <table class="table table-bordered">
   <thead>
    <tr><th>ID</th><th>Nama</th><th>Harga</th><th>Stok</th></tr>
   </thead>
   <tbody>
    <% if (listBarang.length === 0) { %>
     <tr><td colspan="4">Belum ada barang.</td></tr>
    <% } %>
    <% listBarang.forEach(b => { %>
     <tr>
      <td><%= b.id %></td>
      <td><%= b.nama %></td>
      <td><%= b.harga %></td>
      <td><%= b.stok %></td>
     </tr>
    <% }); %>
   </tbody>
  </table>
 </div>
</body>
</html>
```

Variabel listBarang akan dikirim dari rute GET. Kondisi tabel kosong membantu membedakan halaman yang sudah bekerja dengan halaman yang belum menerima data.

2. Di dekat bagian impor app.js, tambahkan Prisma Client dan buat satu instansinya:

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
```

## Formulir tambah barang dan rute GET

3. Buat views/tambah-barang.ejs. Atribut min dan required membantu pengisian di peramban; data tetap diperiksa kembali oleh server pada langkah berikutnya:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Tambah barang</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="container mt-5">
 <h2>Tambah barang</h2>
 <form action="/barang/tambah" method="POST">
  <label for="nama" class="form-label">Nama barang</label>
  <input id="nama" type="text" name="nama" required
   class="form-control mb-2">
  <label for="harga" class="form-label">Harga dalam rupiah</label>
  <input id="harga" type="number" name="harga" min="0" step="1" required
   class="form-control mb-2">
  <label for="stok" class="form-label">Stok</label>
  <input id="stok" type="number" name="stok" min="0" step="1" required
   class="form-control mb-2">
  <button type="submit" class="btn btn-success">Simpan</button>
  <a href="/barang" class="btn btn-secondary">Kembali</a>
 </form>
</body>
</html>
```

4. Tambahkan dua rute GET di app.js sebelum app.listen(...): rute daftar barang dan rute formulir tambah barang.

```
app.get('/barang', async (req, res, next) => {
 try {
  const barangs = await prisma.barang.findMany({
   orderBy: { id: 'asc' }
  });
  res.render('barang', { listBarang: barangs });
 } catch (err) {
  next(err);
 }
});

app.get('/barang/tambah', (req, res) => {
 res.render('tambah-barang');
});
```

findMany() membaca data Barang dari PostgreSQL. Hasilnya diteruskan ke EJS sebagai listBarang. next(err) menyerahkan kegagalan koneksi kepada penanganan galat yang ditambahkan setelah semua rute.

## Menyimpan barang dan memeriksa hasilnya

5. Tambahkan rute POST berikut. Nama wajib diisi; harga dan stok harus berupa bilangan bulat antara 0 dan 2147483647, sesuai batas tipe Int pada skema:

```
app.post('/barang/tambah', [
 body('nama').trim().notEmpty(),
 body('harga').isInt({ min: 0, max: 2147483647 }),
 body('stok').isInt({ min: 0, max: 2147483647 })
], async (req, res, next) => {
 const errors = validationResult(req);
 if (!errors.isEmpty()) {
  return res.status(422).type('text/plain').send(
   'Data tidak valid. Isi nama, harga, dan stok dengan benar.'
  );
 }
 try {
  const { nama, harga, stok } = req.body;
  await prisma.barang.create({
   data: {
    nama,
    harga: Number(harga),
    stok: Number(stok)
   }
  });
  res.redirect('/barang');
 } catch (err) {
  next(err);
 }
});
```

6. Setelah seluruh rute dan sebelum app.listen(...), tambahkan penanganan galat berikut. Pesan teknis dicatat pada terminal, sedangkan peramban menerima pesan singkat:

```
app.use((err, req, res, next) => {
 console.error(err.message);
 res.status(500).type('text/plain').send(
  'Operasi gagal. Periksa koneksi dan pesan pada terminal.'
 );
});
```

7. Jalankan ulang server. Buka http://localhost:3000/barang, klik Tambah barang, lalu simpan data latihan: nama Buku Tulis, harga 12000, dan stok 25.

## Uji hasil

Setelah disimpan, aplikasi kembali ke /barang dan tabel menampilkan satu baris baru dengan ID, nama, harga, dan stok yang sesuai. Tambahkan barang kedua untuk memeriksa pertambahan baris.

Hentikan aplikasi lalu jalankan kembali. Muat ulang /barang: data harus tetap ada selama basis data Supabase masih tersedia. Untuk menguji validasi server, hapus atribut min melalui alat pengembang peramban, kirim harga negatif, dan pastikan tidak ada baris baru yang tersimpan.

## Hasil pembelajaran

create() menyimpan data dan findMany() membacanya kembali. Data tersimpan pada PostgreSQL sehingga tidak hilang saat proses Node.js dijalankan ulang.

LAB 05

# Komponen partial EJS

## Tujuan

Memisahkan struktur header dan footer agar beberapa halaman menggunakan komponen yang sama tanpa menyalin seluruh susunan HTML.

## Langkah praktik

1. Buat folder views/partials jika belum ada. Pertahankan navbar.ejs dari Hari 1 untuk halaman lama. Pada lab ini, gunakan header dan footer baru untuk halaman barang.
2. Buat views/partials/header.ejs dengan isi berikut. Navigasi memakai flex-wrap agar tautan tetap terlihat pada layar sempit:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Aplikasi Node Modern</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body>
 <nav class="navbar navbar-dark bg-dark mb-4" aria-label="Navigasi utama">
  <div class="container flex-wrap gap-2">
   <a class="navbar-brand" href="/">MyApp</a>
   <div class="navbar-nav flex-row gap-3 ms-auto">
    <a class="nav-link" href="/form">Formulir</a>
    <a class="nav-link" href="/barang">Barang</a>
   </div>
  </div>
 </nav>
```

3. Buat views/partials/footer.ejs dengan isi berikut:

```
 <footer class="text-center mt-5 p-3 bg-light">
  <p class="mb-0">
   &copy; <%= new Date().getFullYear() %> Aplikasi Modern.
   Hak cipta dilindungi.
  </p>
 </footer>
 <script
  src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
 </script>
</body>
</html>
```

Header membuka dokumen HTML, memuat Bootstrap, dan menampilkan navigasi. Footer menutup dokumen dan memuat JavaScript Bootstrap. Kedua partial dipanggil satu kali pada setiap halaman yang menggunakannya.

## Menerapkan partial pada halaman barang

4. Buka views/barang.ejs. Hapus bagian awal dari <!DOCTYPE html> sampai tag pembuka <body ...>, lalu ganti dengan dua baris berikut:

```
<%- include('partials/header') %>
<main class="container">
```

5. Pertahankan seluruh isi halaman mulai dari <h2>Manajemen barang</h2> sampai penutup  yang membungkus tabel. Hapus tag penutup </body> dan </html>, lalu ganti dengan:

```
</main>
<%- include('partials/footer') %>
```

6. Lakukan penggantian yang sama pada views/tambah-barang.ejs: ganti bagian awal sampai pembuka body dengan include header dan pembuka main. Pertahankan judul serta seluruh form. Ganti penutup body dan html dengan penutup main serta include footer.

Susunan setiap halaman kini terdiri atas include header, <main class="container">, konten halaman, </main>, lalu include footer. Jangan menyisakan tag html, head, atau body tambahan di halaman pemanggil.

7. Simpan berkas dan buka /barang serta /barang/tambah. Jika server memakai Nodemon, perubahan EJS dapat diperiksa dengan memuat ulang halaman. Jika diperlukan, jalankan ulang node app.js.

## Uji hasil

Pastikan kedua halaman memiliki navigasi dan footer yang sama. Tautan Barang harus menuju /barang, tautan Formulir menuju /form, dan merek MyApp menuju halaman utama dari Hari 1.

Ubah teks MyApp menjadi Inventaris Inixindo hanya pada views/partials/header.ejs. Muat ulang kedua halaman dan pastikan keduanya menampilkan nama baru. Simpan satu barang baru untuk memastikan formulir tetap berfungsi setelah pemisahan HTML.

Gunakan <%- include(...) %> untuk memasukkan HTML partial. Untuk menampilkan nilai data, tetap gunakan <%= ... %> seperti pada nama barang dan pesan kesalahan.

## Hasil pembelajaran

Halaman daftar dan tambah barang berbagi header serta footer. Perubahan navigasi cukup dilakukan pada satu berkas partial, sementara rute dan pengolahan data tetap berada di app.js.

## Pemeriksaan akhir Hari 2

Pastikan formulir menolak input tidak valid, tabel Barang terbentuk, dua data tetap tersimpan setelah server dijalankan ulang, dan halaman barang memakai partial yang sama. Simpan proyek serta package-lock.json; rahasiakan .env.

## Pemecahan masalah praktik

### Formulir tidak terbuka atau req body kosong

Untuk pesan Cannot GET /form, periksa ejaan rute dan pastikan rute didefinisikan sebelum app.listen(...). Jika req.body kosong, letakkan app.use(express.urlencoded({ extended: true })); sebelum rute POST. Nama input pada EJS harus sama dengan nama yang dibaca dari req.body.

### Prisma tidak membaca DATABASE URL

Pastikan .env berada sejajar dengan package.json dan berisi kunci DATABASE_URL. Periksa impor dotenv/config pada prisma.config.ts. Di aplikasi, require('dotenv').config(); harus berjalan sebelum Prisma Client dibuat. Setelah mengubah konfigurasi, jalankan ulang perintah Prisma atau server.

### Koneksi atau sinkronisasi skema gagal

Pastikan proyek Supabase aktif. Gunakan URI Session pooler port 5432 dari Connect, nama pengguna lengkap dengan referensi proyek, serta kata sandi database yang sudah di-encode. Tambahkan sslmode=require. Periksa koneksi internet dan akses port 5432. Lab memakai db push, sehingga tidak memerlukan basis data shadow.

### Prisma Client belum tersedia atau versi berbeda

Jalankan npm ls prisma @prisma/client dan pastikan keduanya versi 6.19.0. Setelah menyunting skema, jalankan npx prisma generate lalu jalankan ulang server. Panduan memakai generator prisma-client-js dengan impor require('@prisma/client').

### Partial tidak ditemukan atau HTML berganda

Periksa lokasi views/partials/header.ejs dan views/partials/footer.ejs. Dari berkas yang langsung berada di folder views, gunakan path partials/header dan partials/footer. Halaman pemanggil hanya menyimpan konten di antara kedua include; struktur html, head, dan body berada pada partial.

### Bahan praktik dan rujukan

Bahan praktik Hari 2: Lab_1_Validasi_Form.md, Lab_2_Setup_Database_Supabase.md, Lab_3_Prisma_ORM.md, Lab_4_CRUD_Operation.md, dan Lab_5_Komponen_Partials.md pada folder Materi_V2/Hari_2.

Supabase, koneksi PostgreSQL: https://supabase.com/docs/guides/database/connecting-to-postgres

Supabase, Prisma: https://supabase.com/docs/guides/database/prisma

Prisma 6, kebutuhan Node.js dan kesesuaian versi paket: https://www.prisma.io/docs/guides/upgrade-prisma-orm/v6

Prisma 6, konfigurasi CLI: https://www.prisma.io/docs/orm/v6/reference/prisma-config-reference